Playing Defense: Improving Blue Team Skills Through Competitions



DMITRIY BERYOZA

ATLSECCON 2026

Slides at https://github.com/0xd13a/presentations

Dmitriy Beryoza https://www.linkedin.com/in/beryozad

- Principal Security Researcher at ***Vectra AI***
- Prior to that - Pentester/Secure Software Development Advocate with X-Force Ethical Hacking Team at ***IBM Security***
- 30+ years in software design and development
- Team lead/challenge builder for ***CyberSci***
- Tech committee/Jury member at ***International Cybersecurity Challenge***
- Presented at SecTor, DEF CON Cloud Village, BSides SF/LV/Ottawa, Hackfest, OWASP meetups
- Ph.D. in Computer Science; OSCP, GREM, CISSP, and others
- *Interests*: reverse engineering, secure software development, CTF competitions



What is this talk about?

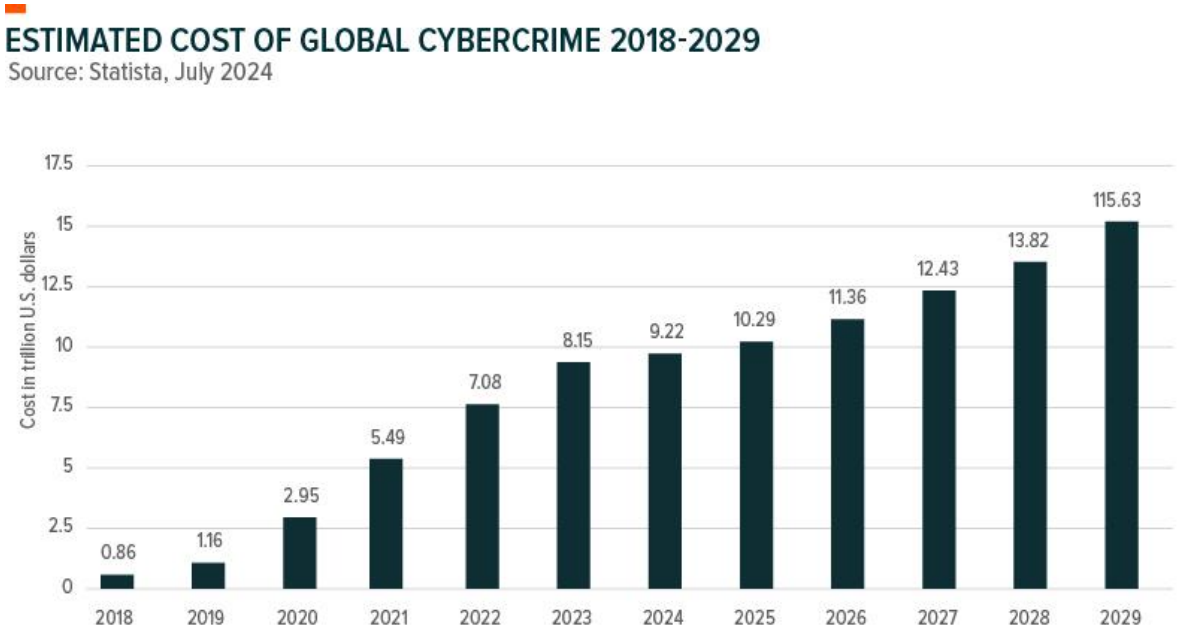
- I would like to discuss the state of developers' cybersecurity skills and how they can be improved through practical security exercises
  - In particular – through gamified training and competitions

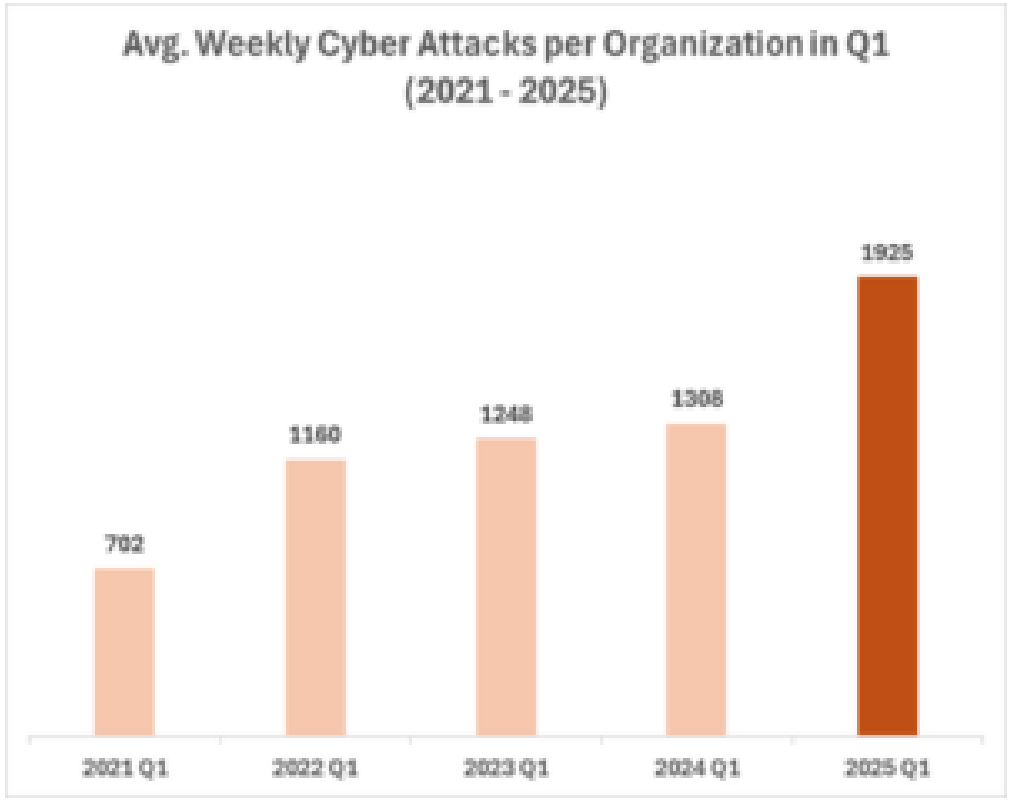
- For the purposes of this presentation, I will use "blue-teamers", "defenders", "developers" interchangeably
  - Similar set of cybersecurity skills and knowledge is required of anyone who is building, configuring and defending software systems
  - This includes developers, architects, Quality Assurance, IT, DevOps, DevSecOps



Cyber attacks are growing

- Despite increased attention paid to cybersecurity, problems persist
- Cyber attacks increase steadily year/year
- Multiple reported daily incidents in cybersecurity news
- Cost of cybercrime keeps growing as well



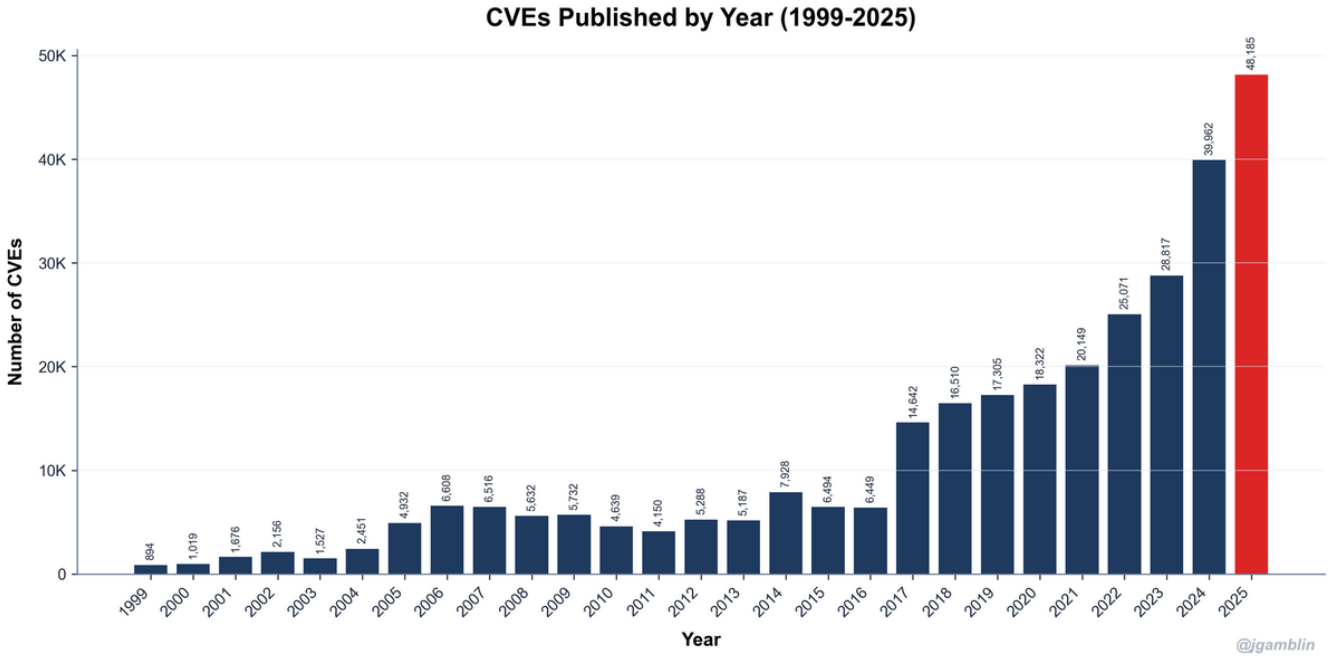


Q1 2025 Global Cyber Attack Report from Check Point Software

Role of software vulnerabilities in cyber incidents

- There are many factors that enable cyberattacks
  - Software vulnerabilities
  - Software misconfigurations
  - Human factors (e.g. social engineering)
  - Anonymity/jurisdictional fragmentation
  - …and others

- Software vulnerabilities are arguably the biggest contributor
  - Without them other factors would be far less effective
  - Despite our growing knowledge of how to develop secure software, real software quality does not show improvement
  - CVE rates increase significantly year/year



https://www.thestack.technology/cves-in-2025-analysis/

Why does it keep happening?

- There are multiple reasons for software vulnerabilities persisting
  - See great discussion in Bruce Schneier's books, e.g. "Secrets and Lies"
- Incentive misalignment
  - Vendors are rewarded for new features, usability, speed of delivery over security
- Lack of liability
  - No legal or financial liability for vulnerabilities in software
- Users are forgiving of security defects and rarely compare products based on security
  - Defects are expected to be patched after the fact, as they are being discovered
- Security is a "negative" property
  - You can't easily see or prove security of software
- Insufficient developer cybersecurity skills
  - Let's discuss this one in greater detail



Insufficient developer knowledge and skills

- Many developers today have insufficient knowledge of building secure software
  - Often vulnerabilities are introduced not by omission, but because of fundamental knowledge gaps
  - I have observed this when I was a developer
  - I have experienced this when I was working as a pentester, examining code for security vulnerabilities
  - One can see this by examining details of CVEs logged for open-source products
- Lack of training is the primary culprit
  - Most college-educated developers from 90s and 2000's didn't study cybersecurity in school
  - Many developers are self-taught (roughly 1/3 based on surveys)
  - Developers entering workforce today are taking advantage of LLMs and vibe coding, which does not increase their knowledge
- Today Computer Science education has improved, but there are still issues
  - Dedicated cybersecurity degrees exist but they are not for regular developers
  - In many CS programs cybersecurity subjects are optional

Won't AI solve this for us?

- LLM have been trained on all kinds of code, good and not so good
  - That learning translates into the quality of code samples they generate
  - 45% of generated code contains security vulnerabilities [Veracode 2025]
- Vibe coding produces even worse results [Tenzai study in Dec. 2025]
  - 69 vulnerabilities across 15 apps
  - 100% of apps had serious vulnerabilities
  - 0% implemented basic protections
- In a rush to produce new technology corners are cut and not enough thought is given to security
  - "The industry created a brand-new executable format written in plain human language and forgot every control that should come with it." [J.O'Reilly, security adviser to the OpenClaw project]
- AI improvements have not eliminated the need for developers to have good cybersecurity knowledge
  - Humans still need to be in the loop to verify what AI creates
  - Humans need to know what “secure” looks like



Obstacles to acquiring security knowledge

- Practicing secure software development and testing software for vulnerabilities is not trivial
  - Other types of defects (UI, performance, resource consumption) are easier to understand and verify
  - You often need an (expensive) dedicated security professional to act as a counterpart in discovery and testing
- Verification of software security is sometimes offloaded to later release stages
  - Vulns are found during security code audits, pentests, red team engagements, bug bounties, vuln reports
  - This is an expensive way of finding issues, and done too late in the game (if ever)
- There is an inherent imbalance between focus and motivation of attackers vs. defenders

Dimension	Red teamers	Developers
Mindset	Break software	New features, good user experience, on-time delivery
Focus on security	High	Medium/Low
Security knowledge	High	Medium/Low
Vulnerability coverage	Targeted vulnerability discovery	Need to protect an entire attack surface

How can we improve the situation?

- Despite these obstacles we should still try to make things better
- Continuous developer education is highly beneficial
  - If your developers lack secure software development knowledge skills, you should invest in training
  - Online training, videos, books on security will all make a difference
- As with all learning, however, acquired knowledge should be practiced
  - Active learning improves retention and reduces failure rates in STEM fields [Wikipedia]
  - Simply reading the information is not as effective
- It also helps to have learning experiences gamified (through competing against others, points, rewards)
  - Research shows that it improves engagement, motivation and performance [The Gamification of Learning: a Meta-analysis]
  - Also moderately improves knowledge retention

Available practical cybersecurity learning

- The good news: plenty of opportunities exist for practicing cybersecurity knowledge and skills
- CTF (capture the flag) competitions (Jeopardy and Attack/Defense)
  - CTFTime.org (100s of CTFs/year); competitions at security conferences; internal company events; international competitions
- Practice sites: Hack the Box, TryHackMe, Vuln Hub
- Challenge sites (Wechall.net)
- Practical security certificates (e.g. OSCP)





Available practical cybersecurity learning (cont.)

- The bad news: majority of these opportunities exercise ***offensive*** security knowledge
- Although it deals with software vulnerabilities, it's a very different skillset
- CTF challenge categories are primarily about bug exploitation
  - Reverse engineering, binary exploitation, crypto, and others
- Despite the name, Attack/Defense is also mostly about exploiting vulnerabilities
- These learning opportunities are not good matches for training builders
  - Not relevant/intimidating –they exercise knowledge developers don't have and don't need
  - Looking back to my days as a developer – I would not have found them useful, and would have lacked the necessary knowledge

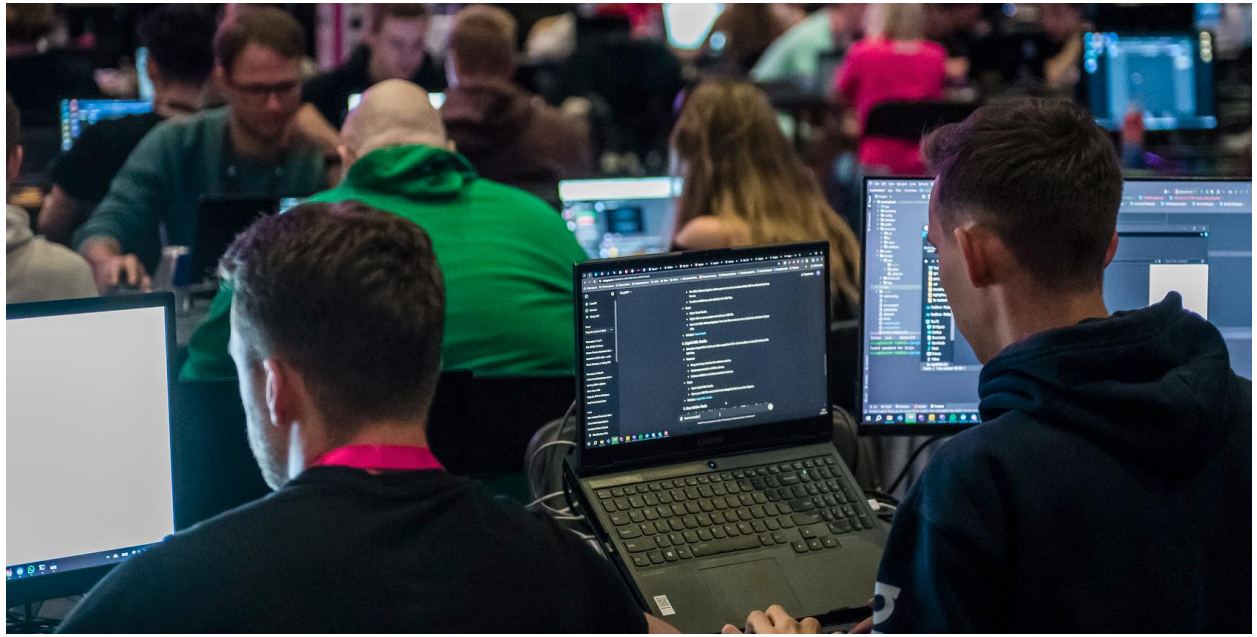
What could help?

- A **defense-style competition**, where participants analyze and fix vulnerabilities in systems that are continuously under attack

- A live environment where defenders can analyze and fix software
  - Practicing skills makes for a more effective learning process compared to passive learning from books or presentations

- An automated way to conduct attacks
  - Vulnerabilities must be exercised to check if they have been fixed

- A gamified experience to motivate participants and make learning fun
  - Scoring of results and competition between participants serves as an additional motivating factor



- Defense competitions & environments do exist but they are not nearly as common
  - Fix The Flag (DEF CON AppSec Village), Locked Shields, DEF CON Blue Team Village CTF, OpenBAS, and some others

Additional benefits of defense competitions

- Required forensic analysis
  - In real life attacks defenders must first understand how the attack is being conducted
  - This involves analysis of packet captures, logs, and user requests
- Learning to analyze unfamiliar code
  - Once one figures out how the attack is conducted, the first impulse is to block it (e.g. via firewall, WAF, IPS rules)
  - The better fix is to analyze vulnerability in the code and fix it
- Timeboxing
  - Time pressure is an additional motivator and simulates challenges of real-life incident response
- Maintaining uptime
  - Any changes made to the system you are defending cannot break legitimate functionality, and uptime must be maintained to continue to serve customers

Siege

- We recently released **Siege** – an open-source defense competition framework
  - Built as part of my work with CyberSci, a Canadian cybersecurity non-profit
  - We have used it to host defense-style competitions for the past 5 years
  - We hope it will be useful to anyone who is training developers and defenders

- Software can be found at:

https://siege.software

https://github.com/0xd13a/siege



PSA: Canada's Cyber Security Challenge

- National cybersecurity competition for university students (running since 2010)
- Regional and national events (60+ teams), winners represent Canada internationally
- Team Canada competed in Prague, Athens, Vienna, San Diego, Santiago, Tokyo



Join us! Become a:

- ***Sponsor*** - help us grow the next generation of cyber professionals
- ***Employer*** - interview and hire the brightest students in the field
- ***Organizer*** - join our team to organize events and build challenges
- ***Player*** - start or join your university team



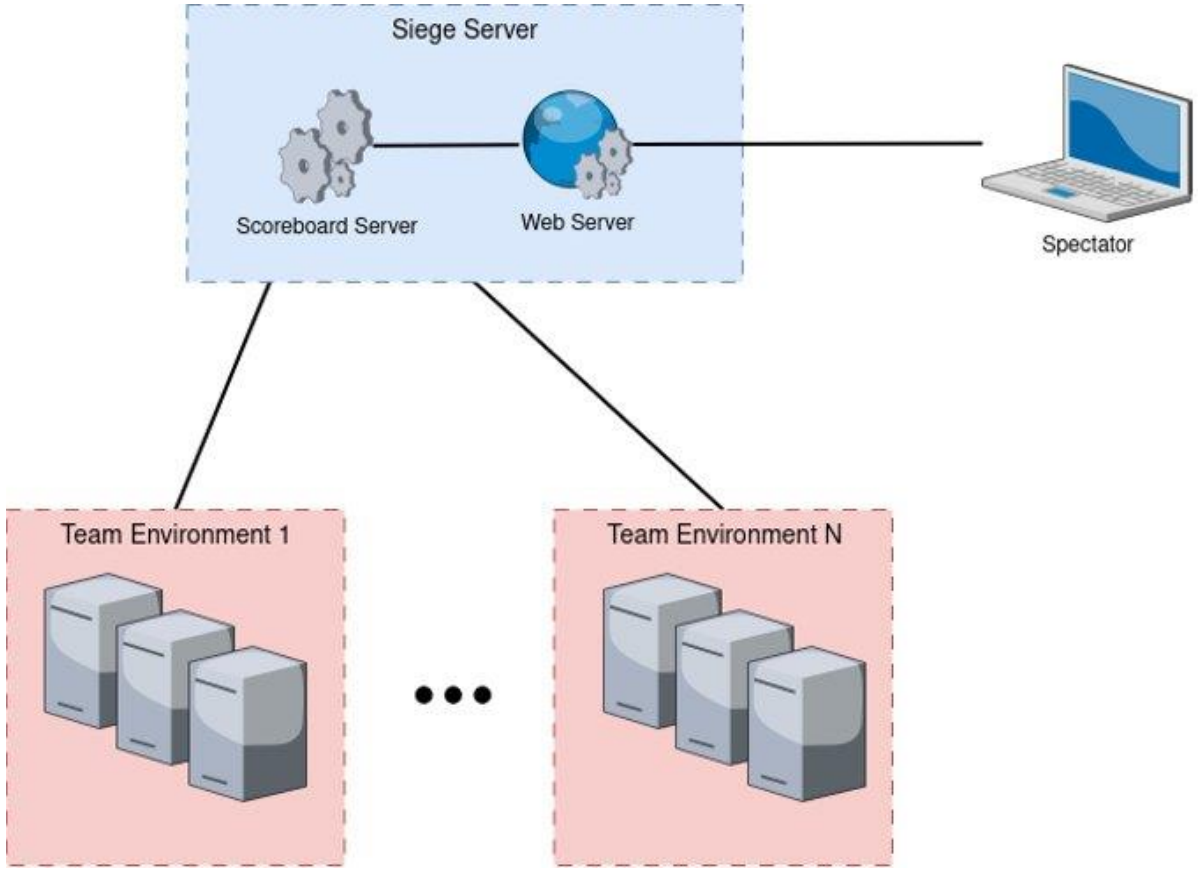
https://cybersecuritychallenge.ca/

How does it work?

- During the competition, every team get access to a set of vulnerable services
- Competition environment periodically sends series of requests to those services
  - Malicious requests exploit the vulnerabilities
  - Benign requests make sure the service functionality is working as intended
- Players study services and the incoming traffic
  - They try to fix vulnerabilities to make malicious requests fail
  - Benign requests must continue working correctly to maintain service functionality
- Teams are rewarded/penalized with points; the one with the highest score wins

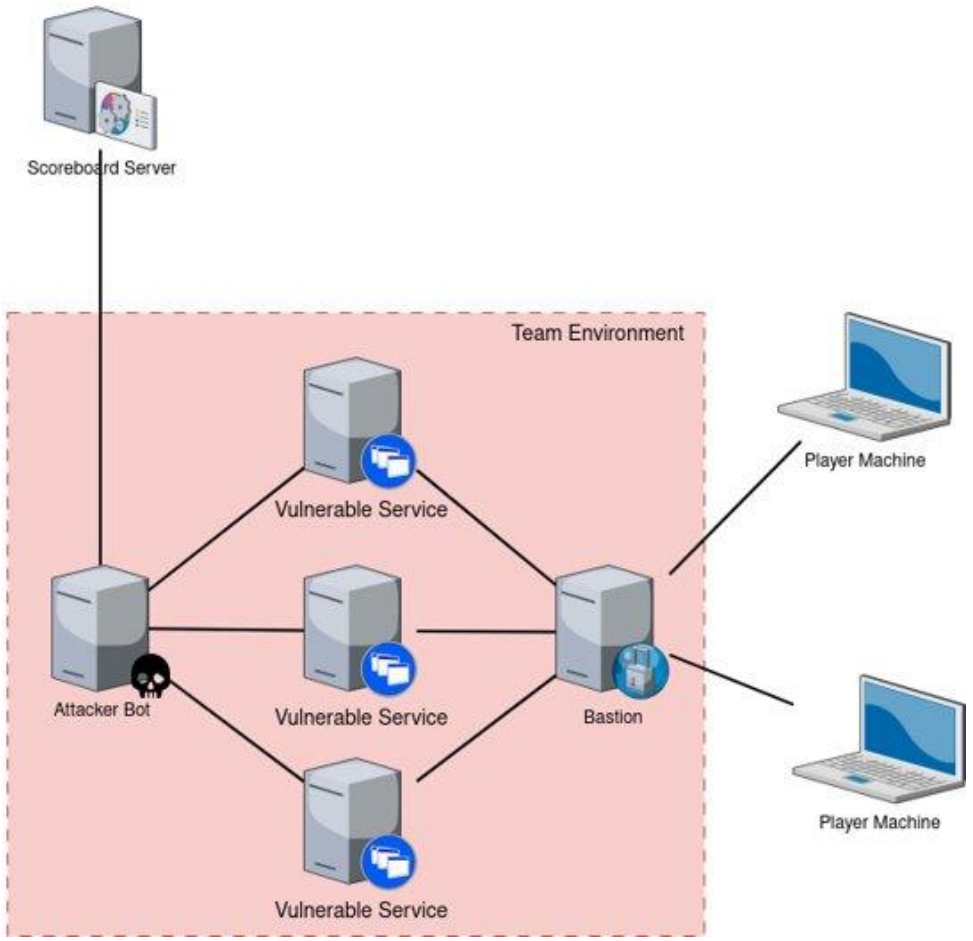
General architecture

- *Scoreboard* server controls the competition flow and manages the scoreboard
- *Web server* displays competition scoreboard and individual teams' progress
- Each team has an *environment* in which attacks execute and players fix service vulnerabilities



Team environment

- Every team has VPN access through a *bastion host* into a small network of VMs
- An *attacker bot* runs attack scripts, one per service. Each executes series of benign and malicious requests against vulnerable service on the network. Bot reports results to *scoreboard server*
- Players have full control of machines hosting vulnerable services and try to discover and fix security vulnerabilities



Competition timeline

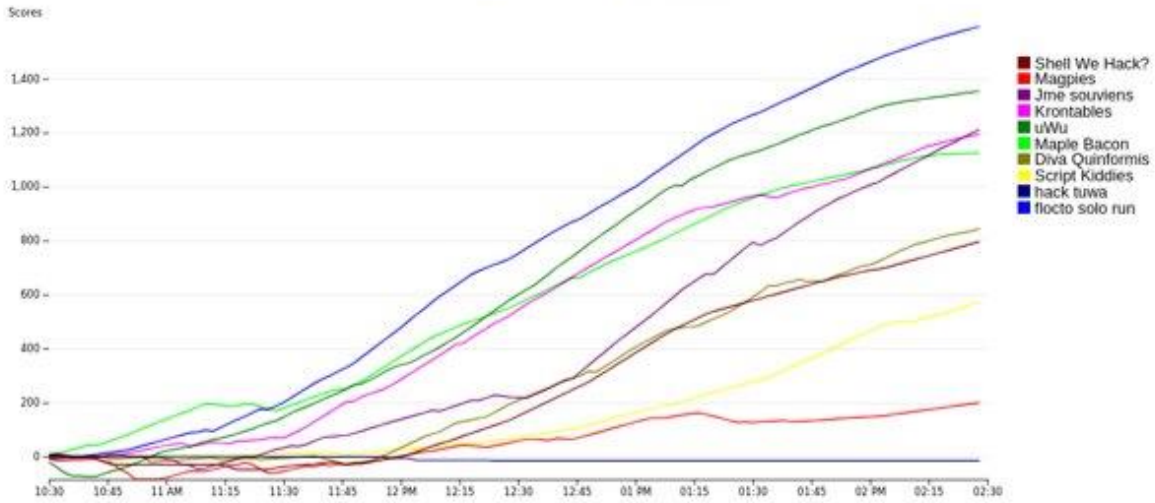
- Study period (T-1hr)
  - Teams are given access to their subnets
  - Study period is spent analyzing services and initiating fixes
- Competition clock starts (T+0h)
  - Competition is divided in a series of ticks (typically 2 minutes)
  - Every tick a series of requests, benign and malicious, are sent to the services
  - Teams capture incoming traffic, trying to determine what exploits are sent
  - Vulnerabilities are continuously fixed, services are rebuilt
- Competition ends (T+4h)
  - 4 hours seems to be a sweet spot to fix 3-5 services, longer may be tiring
  - Winners are determined

Scoreboard

CyberSci

CyberSci Nationals 2025

Competition has finished.



Place	Team	Score
1	flocto solo run	1594
2	uWu	1354
3	Jme souviens	1212
4	Krontables	1194
5	Maple Bacon	1126

Time	Tick	Fuel Management Service						
		😀	😐	😈	🛡	💀	😎	Σ
5/31/2023, 11:43:00 PM	168	5			5			10
5/31/2023, 11:42:00 PM	167	5			5			10
5/31/2023, 11:41:00 PM	166	5			5			10
5/31/2023, 11:40:00 PM	165	4	-1		5			8
5/31/2023, 11:39:00 PM	164	5			5			10

Scoring

- We score each request as follows (values are configurable):
  - Successful benign request - +1 point
  - Failed benign request - -1 point
  - Successful malicious request - -1 point
  - Failed malicious request - +1 point
  - Service down - -1 point
- We usually send an equal number of both types of requests per tick (e.g. 5 benign and 5 malicious)
  - This way the points for the service stay at 0 if nothing changes
- Request combinations can be configured with a schedule to add variability and unpredictability
- Scores can be capped to avoid “runaway” winners (a team finishing fixes early and accumulating positive points indefinitely)

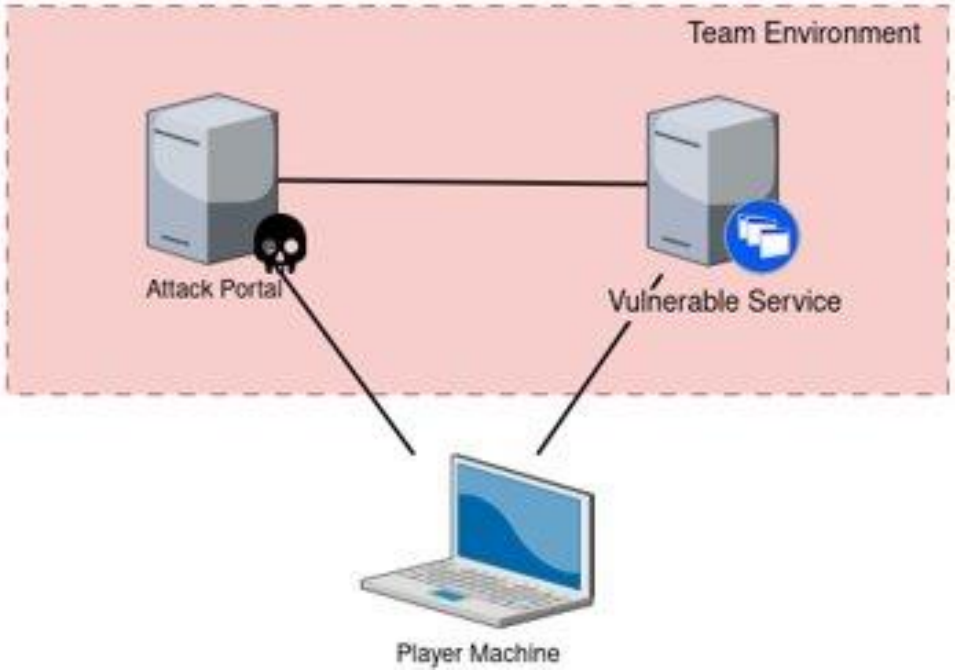
Building vulnerable services

- Each service has a companion attack script developed, which sends benign and malicious requests
- Most services are available in source code, but some can require RE
- We usually host services on separate VMs to ease of management
  - Putting service in a container inside the VM facilitates building and deploying code
- Pretty much any technology can be used in a service:
  - Traditional website technologies
  - Common protocols (LDAP, IMAP, DNS, etc.)
  - Bespoke APIs
  - Terminal software
  - GUI apps
  - Misconfigured cloud services, environments, firewalls
  - AI chatbots



Siege Light

- In some cases, building and running an entire competition is too much work
- We provide ability to run challenges in “light” mode that does not require a central scoreboard server or attack bots
- Teams get simple environments consisting of an attack portal machine and a set of vulnerable services
- Players drive attacks by interacting with an attack portal
- Attack is a series of requests, some of which exploit a particular vulnerability



Siege Light (continued)

- Player analyzes requests and tries to fix the vulnerability
- If vulnerability is fixed, the attack fails and portal reveals a flag
- Flag can then be submitted to a traditional Jeopardy competition scoreboard (e.g. CTFd)

Future work

- We are planning to evolve Siege functionality:
  - GUI for competition configuration and management
  - Support for a production-quality WSGI server like Gunicorn
  - Isolation for attacker scripts
  - AWS support for competition environment
  - Option to host some of the infrastructure in containers
- How to contribute?
  - Questions, ideas, bug fixes and new features are welcome!
  - Log and tag issues to communicate
  - For small fixes: submit a PR; maintain style, document, update unit tests
  - For bigger features: let’s discuss first

Summary

- Deficiencies in developer knowledge and skills play a significant role in proliferation of security vulnerabilities
- Filling those knowledge gaps is not easy because of a variety of factors
- Practical training in identifying and fixing vulnerabilities can improve developer knowledge and increase software quality
- Competitions centered around defending and patching vulnerable software are a particularly effective way of learning to build secure software



Q&A

Thank you for listening and please

reach out with any questions!

https://www.linkedin.com/in/beryozad/



https://siege.software

https://github.com/0xd13a/siege